

UNIT 4:

MEMORY MANAGEMENT

[14 Marks]

Hamid Ali

Lecturer, Computer Engg.

A.I. ARKP, New Panvel

4

TLO 4.1 Compare fixed and variable memory partitioning.
TLO 4.2 Differentiate between Bit map and Linked list technique.
TLO 4.3 Explain working of various partitioning algorithm.
TLO 4.4 Calculate page fault for given page reference string.

Unit - IV Memory Management

4.1 Basic Memory Management: Partitioning - Fixed and Variable, Free Space Management Techniques: Bit map, Linked List
4.2 Swapping, Compaction, Fragmentation, Partitioning Algorithms: First fit, Best fit, Worst fit
4.3 Non-contiguous Memory Management Techniques: Paging, Segmentation
4.4 Virtual Memory: Basics, Demand paging, Page Fault
4.5 Page Replacement Algorithm: First In First Out (FIFO), Least Recently Used (LRU), Optimal

Memory Management in OS

User Programs

User Interface

Operating
System

Systems Calls

Operating
System

File
Management

Memory
Management

Process
Management

Network
Management

Power
Management

Hardware

Basics of Memory Management

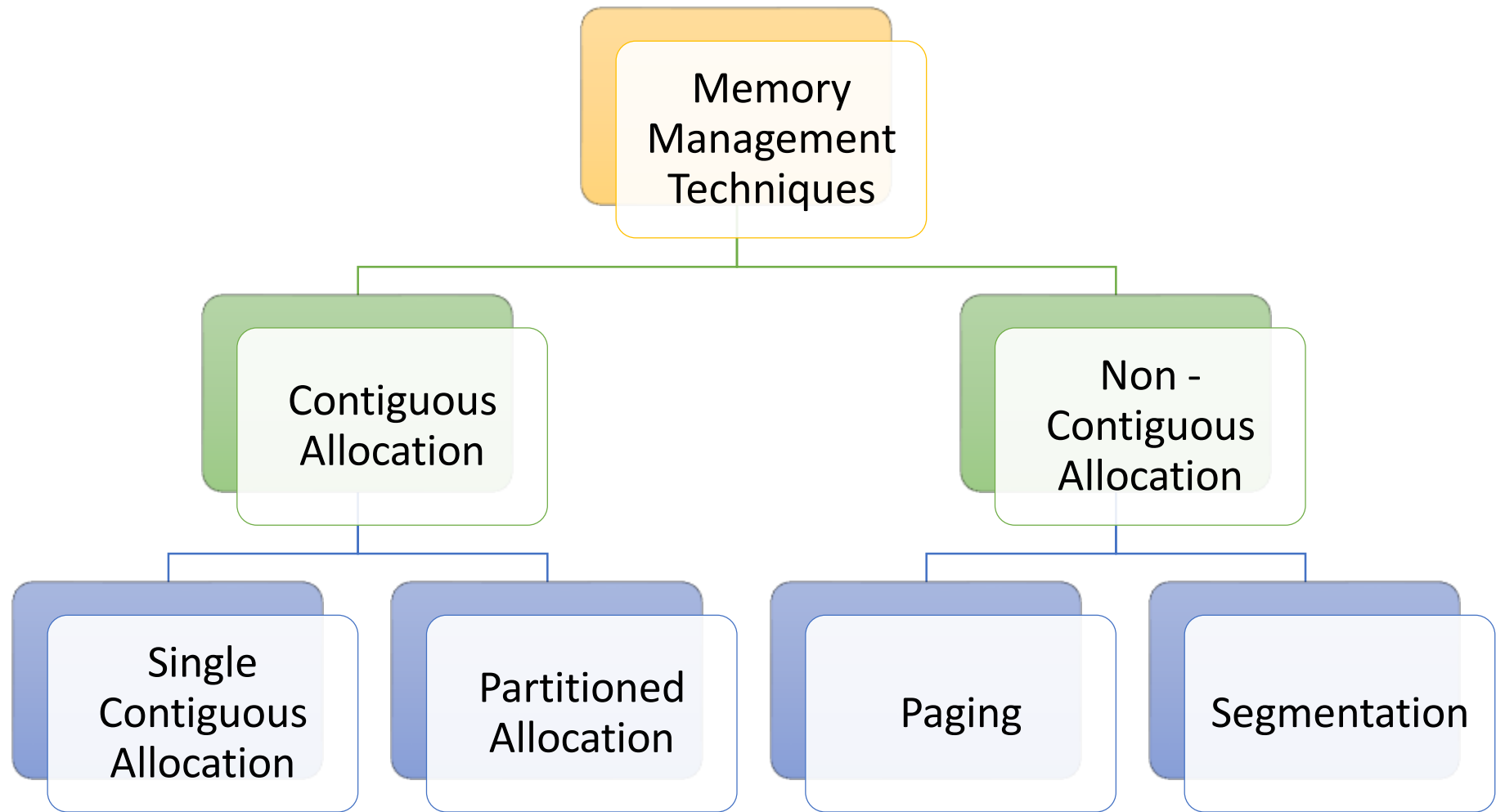
- Memory management is an essential component in operating systems.
- It deals with the usage of the primary memory of the computer system.
- Its purpose is to move processes between primary memory (R.A.M.) and secondary memory (R.O.M.).
- Most of the operating systems these days make use of memory management. It increases the efficiency and performance of the operating system.

Functions of Memory Management

- **Process Isolation:** Process Isolation is the mechanism of keeping processes separate so that one process cannot access or modify another process's memory without permission.
- **Tracking of Memory Locations:** Tracking of memory locations is the process of keeping a record of the memory areas that are currently in use and those that are free.
- **Automatic Allocation and Management:** Automatic Allocation and Management means the OS automatically allocates and manages memory for processes without requiring the user to manage it manually.
- **Long Term Storage:** Long-term storage is used to permanently store programs, files, and data for future use.
- **Support of Modular Programming:** Modular programming means dividing a large program into small, separate modules (functions or parts). Each module performs a specific task.
- **Protection and Access Control:** Protection and Access Control means controlling who can access which resources of a computer system and what actions they can perform.
- **Keeping status of Main Memory Locations:** The OS maintains this information so that it can allocate memory to new processes and release memory when processes finish.

What is Memory Management?

- Memory Management is the process of controlling and coordinating computer memory, assigning portions known as blocks to various running programs to optimize the overall performance of the system.
- It is the most important function of an operating system that manages primary memory.
- It helps processes to move back and forward between the main memory and execution disk.
- It helps the Operating System (OS) keep track of every memory location, whether it is assigned to a process or still free.



Contiguous Memory Allocation

Contiguous Memory Allocation is a memory management technique in which an entire process is loaded into a single continuous block of main memory.

Non-Contiguous Memory Allocation

Non-Contiguous Memory Allocation is a memory management technique in which a process is divided into smaller parts and stored in different locations in main memory.

Single Contiguous Allocation

- It is the easiest memory management technique.
- In this method, all types of computer's memory except a small portion which is reserved for the OS is available for one application.
- For example, MS-DOS operating system allocates memory in this way. An embedded system also runs on a single application like Car control system and Washing machine.

Partitioned Allocation

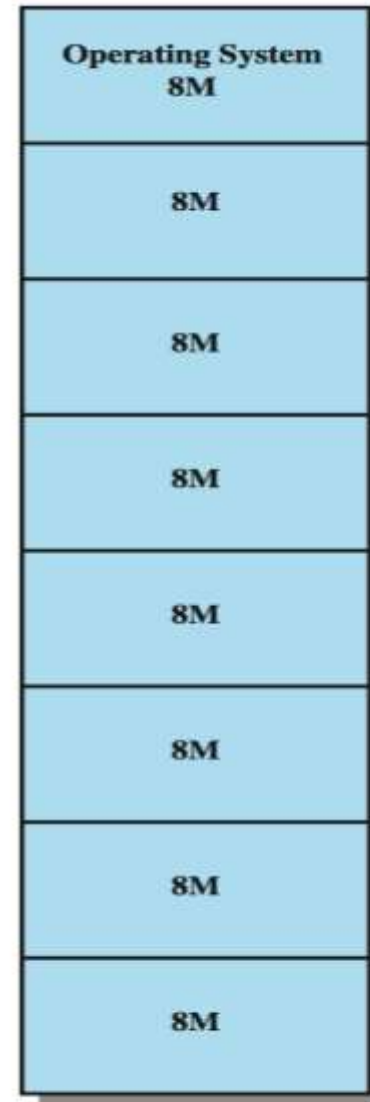
- It divides primary memory into various memory partitions, which is mostly contiguous areas of memory.
- Every partition stores all the information for a specific task or job.
- This method consists of allotting a partition to a job when it starts & unallocated when it ends.

Two Methods:

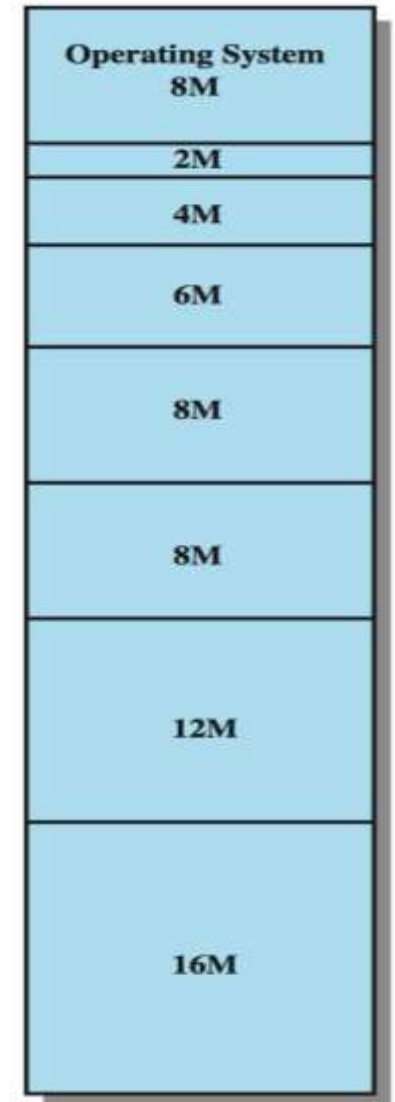
- Static (Fixed Sized) Memory Partitioning
- Dynamic (Variable Sized) Memory Partitioning

Fixed Partitioning

- This is the oldest and simplest technique.
- In this partitioning, the number of partitions (non-overlapping) in RAM is **fixed** but the **size** of each partition may or **may not be the same**.
- Partitions are made before execution or during system configure.



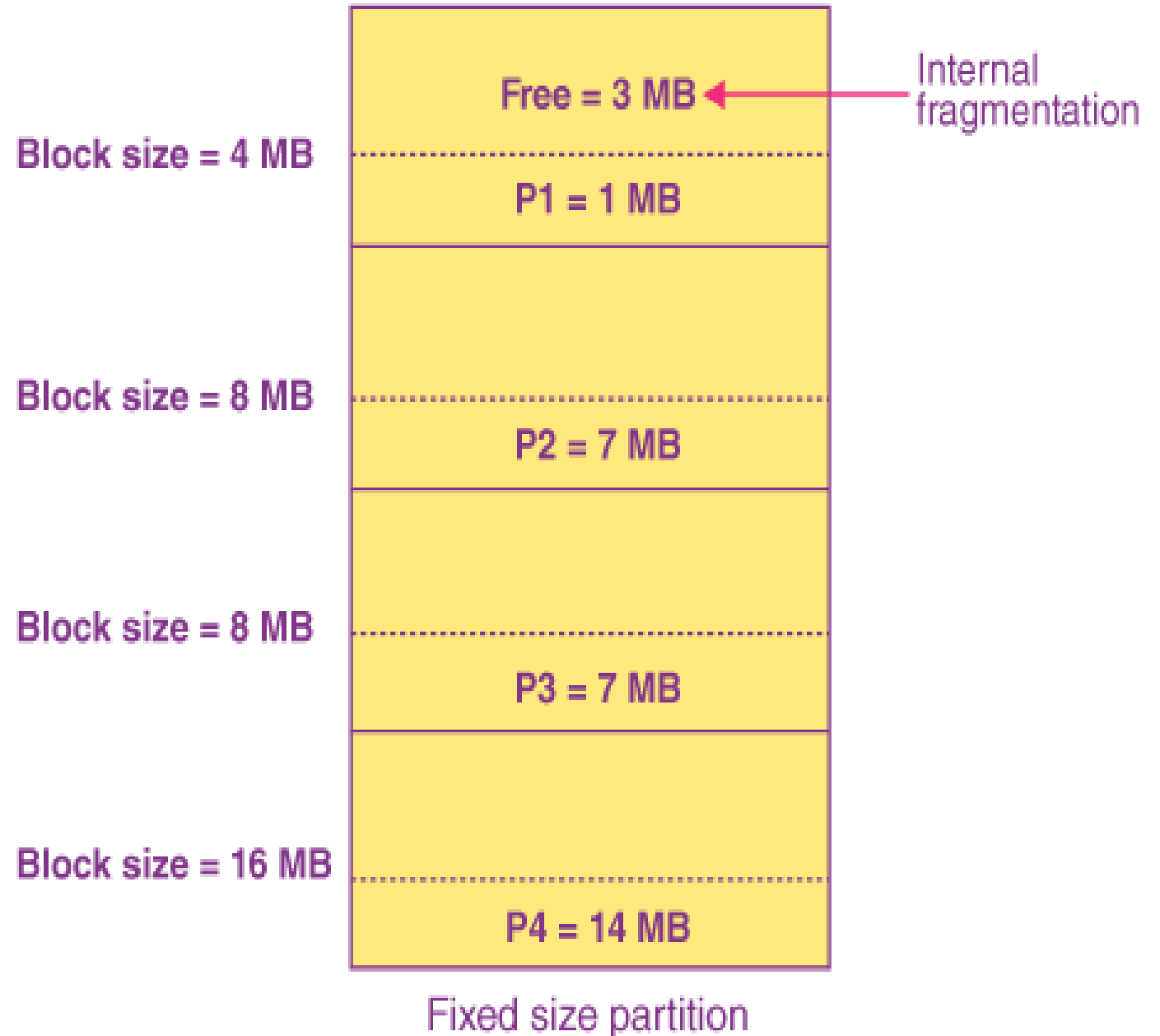
(a) Equal-size partitions



(b) Unequal-size partitions

Internal Fragmentation

Internal fragmentation occurs when a process is smaller than the partition assigned to it, leaving some memory inside that partition unused and cannot be used by another process.



Static OR Fixed Partitioning

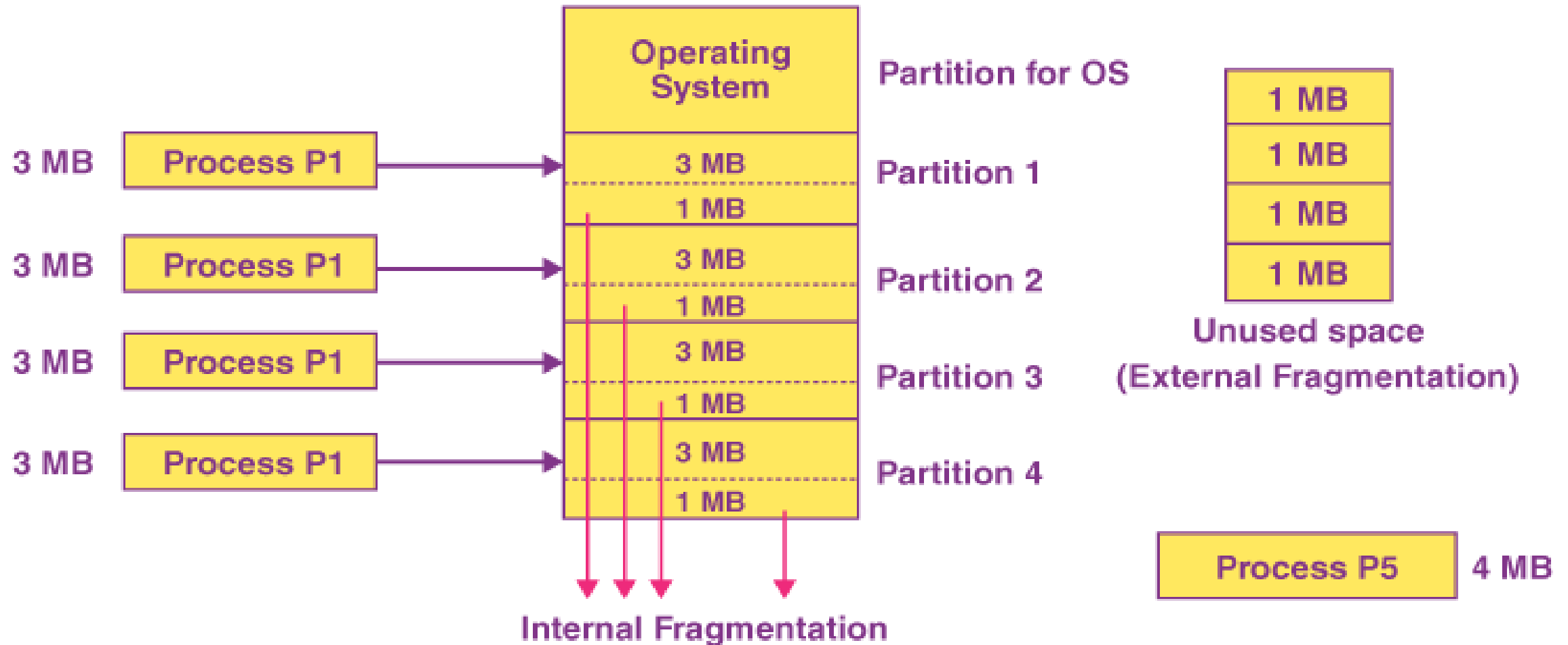
As you can see in the figure illustrated above, the very first process here consumes only 1MB in the main memory out of 4MB.

The internal fragmentation in first block = $(4-1)$ MB = 3 MB

The sum of internal fragmentation in all the individual blocks
= $(4-1)$ MB + $(16-14)$ + $(8-7)$ MB + $(8-7)$ MB
= 3 MB + 2 MB + 1 MB + 1 MB
= 7MB

Suppose that the process P5 has a size of 7MB. Now, this process can't get accommodated even if there's free space available since we have contiguous allocation (because spinning won't be allowed).

Thus, 7 MB would become part of external fragmentation.

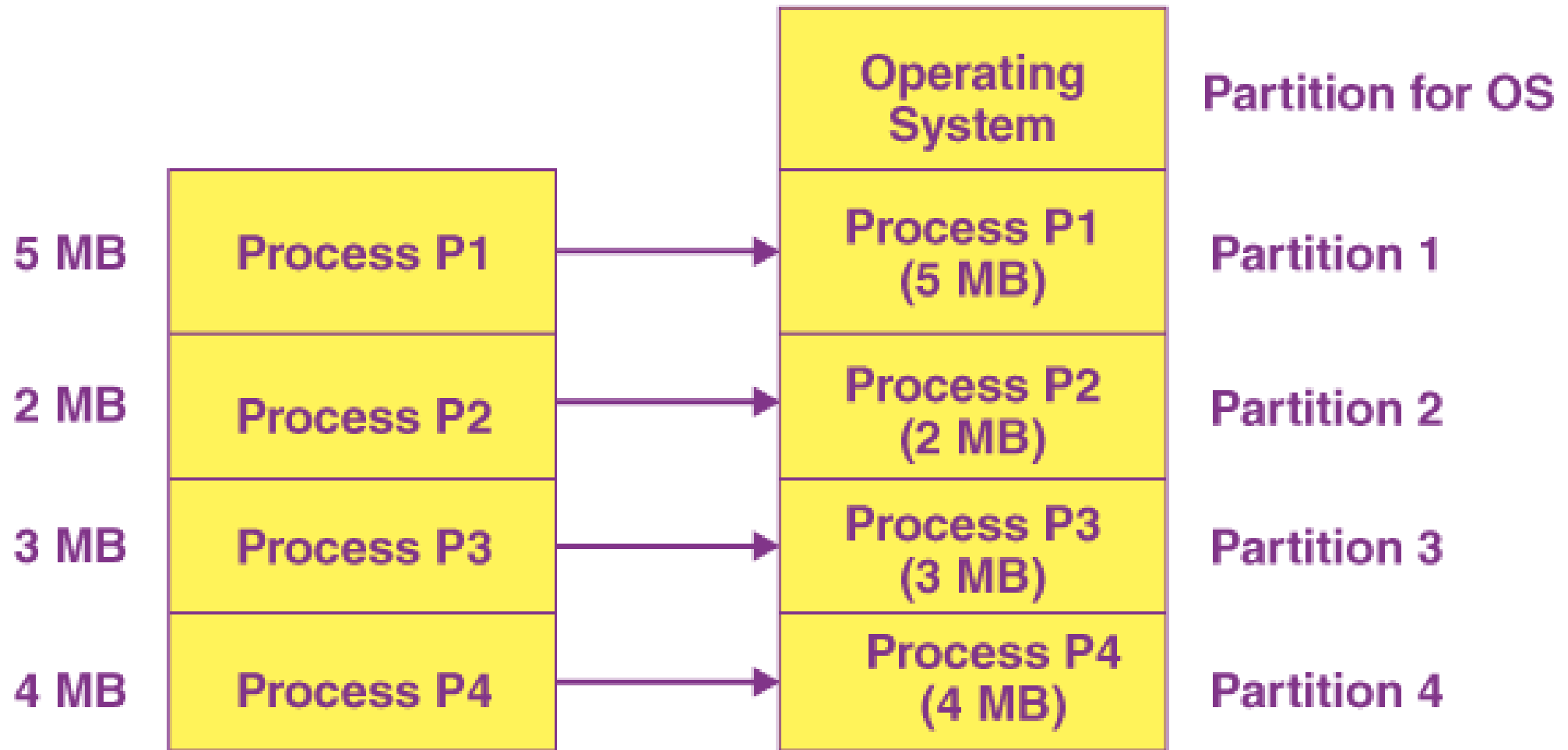


Fixed Partitioning

(Contiguous memory allocation)

Dynamic Memory Partitioning OR Variable Sized Partitioning

- Dynamic partitioning helps in overcoming the difficulties caused by the process of fixed partitioning.
- In dynamic partitioning, the partition size initially isn't declared. It's declared at the moment of process loading.
- The very first partition has to be reserved for the OS.
- The left space gets divided into various parts.
- The actual size of every partition would be equal to the process size.
- The partition size is made according to the memory requirement of the process. Therefore, very little or no memory is wasted inside the partition.



Dynamic Partitioning

(Process Size = Partition Size)

Dynamic Memory Partitioning OR Variable Sized Partitioning

W16,W18

- When a process enters in main memory, it is allocated exact size that is required by that process. So in this method, partitions can vary in size depending on memory space required by a process entering in main memory.
- Operating system maintains a table indicating which parts of memory are available and which are occupied.
- When new process arrives and it needs space, system searches for available memory space in main memory. If it is available then memory is allocated to the process by creating a partition in memory.
- Like this depending on size of process and available memory, partitions take place in main memory.

Dynamic Memory partitioning

➤ **Advantages:**

- No internal fragmentation.
- More efficient use of main memory.

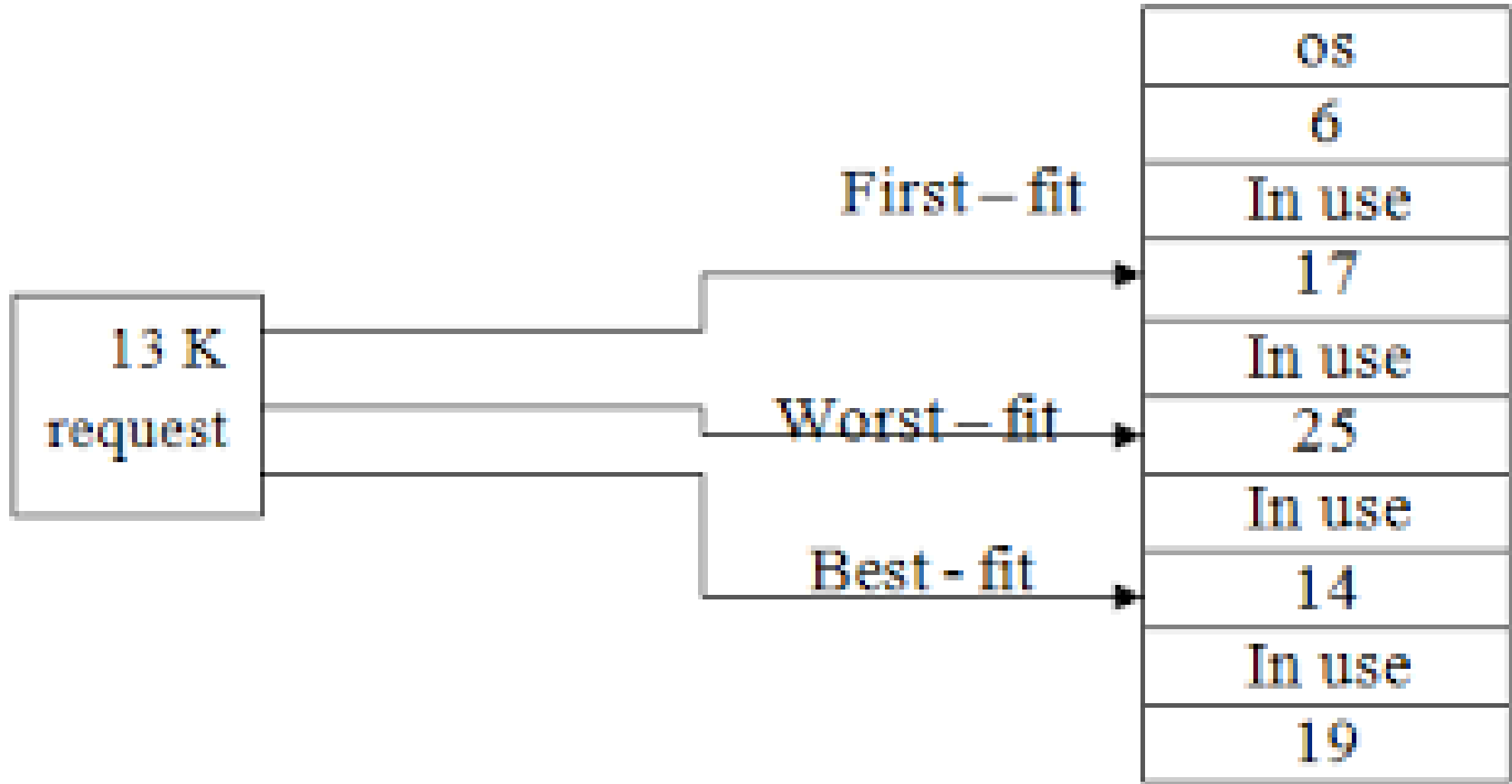
➤ **Disadvantages:**

- It suffers from external fragmentation.
- It needs compaction

(Compaction = Combining small free memory spaces into one large free space.)

Below are the various partition allocation schemes :

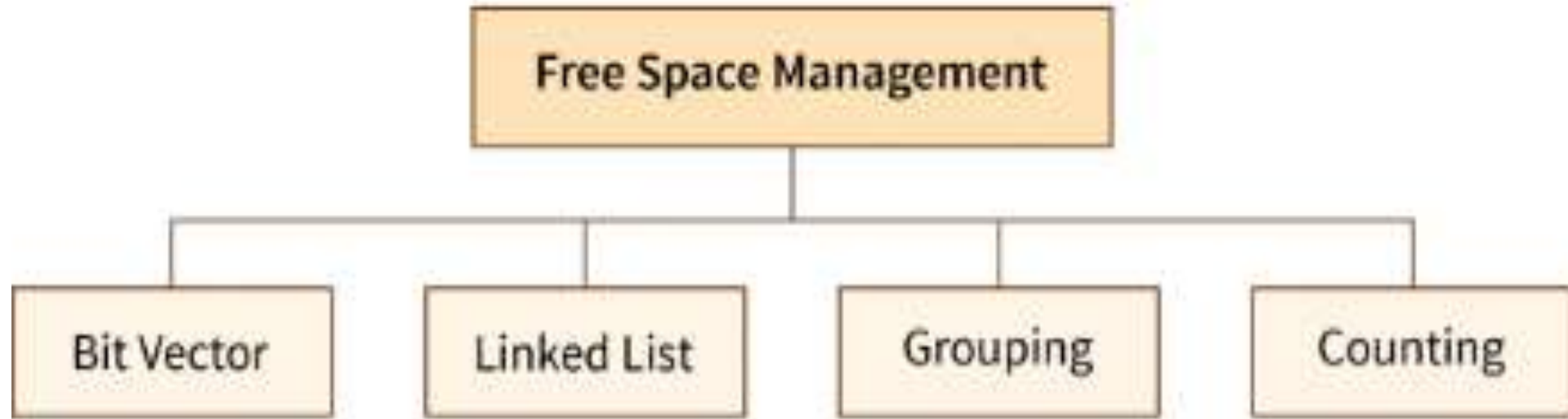
- **First Fit:** In this type fit, the partition is allocated, which is the first sufficient block from the beginning of the main memory.
- **Best Fit:** It allocates the process to the partition that is the first smallest partition among the free partitions.
- **Worst Fit:** It allocates the process to the partition, which is the largest sufficient freely available partition in the main memory.



Basis	Fixed–Size Partition	Variable–Size Partition
Size of Partition	OS decides the size of the partition only once during the system boot time. It can't be changed.	OS decides the size of the partition according to the size of the process. The size of the partition can be changed.
Degree of Multiprogramming	Fix the degree of multiprogramming.	The degree of multiprogramming will vary according to the size of the processes.
Fragmentation	It supports internal fragmentation.	It supports external fragmentation.
Another name	Static partitioning.	Dynamic partitioning.
Uses	IBM/360, DOS and OS/MFT operating systems use this scheme.	IBM, OS/MFT uses this scheme.

Free Space Management Techniques

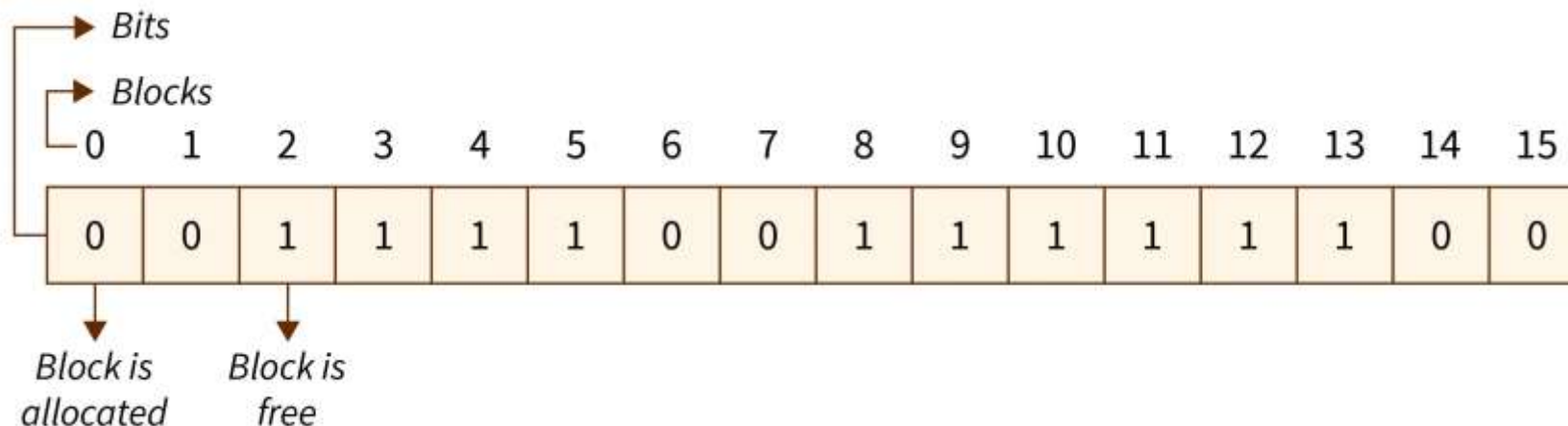
- There is a system software in an operating system that manipulates and keeps track of free spaces to allocate and de-allocate memory blocks to files, this system is called a "file management system in an operating system". There is a "**free space list**" in an operating system that maintains the record of free blocks.
- When a file is created, the operating system searches the **free space list** for the required space allocated to save a file. While deletion a file, the file system frees the given space and adds this to the "**free space list**".



Bitmap or Bit Vector :

- A Bitmap or Bit Vector is series or collection of bits where each bit corresponds to a disk block. The bit can take two values 0 and 1:
- The free-space list is implemented as a bit map or bit vector.
- 1 indicates that the block is free and 0 indicates an allocated block.
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13 are free and the rest of the blocks are allocated.

The free-space bit map would be : 0011110011111100



Bitmap or Bit Vector :

- A bit vector is the most frequently used method to implement the free space list.
- A bit vector is also known as a "**Bit map**".
- It is a series or collection of bits in which each bit represents a disk block.
- The values taken by the bits are either 1 or 0.
- If the block bit is 1, it means the block is free and if the block bit is 0, it means the block is not free (occupied).

Bit Map

- **Advantages of Bit vector method :**

- 1.Simple and easy to understand.
- 2.Consumes less memory.
- 3.It is efficient to find free space.

- **Disadvantages of Bit vector method :**

- 1.The operating system goes through all the blocks until it finds a free block.
- 2.It is not efficient when the disk size is large.

Linked List :

- A **linked list** is another approach for free space management in an operating system. In it, all the free blocks inside a disk are linked together in a "**linked list**".
- These free blocks on the disk are linked together by a pointer.
- These pointers of the free block contain the address of the next free block and the last pointer of the list points to null which indicates the end of the linked list.
- This technique is not enough to traverse / search the list because we have to read each disk block one by one which requires I/O time.

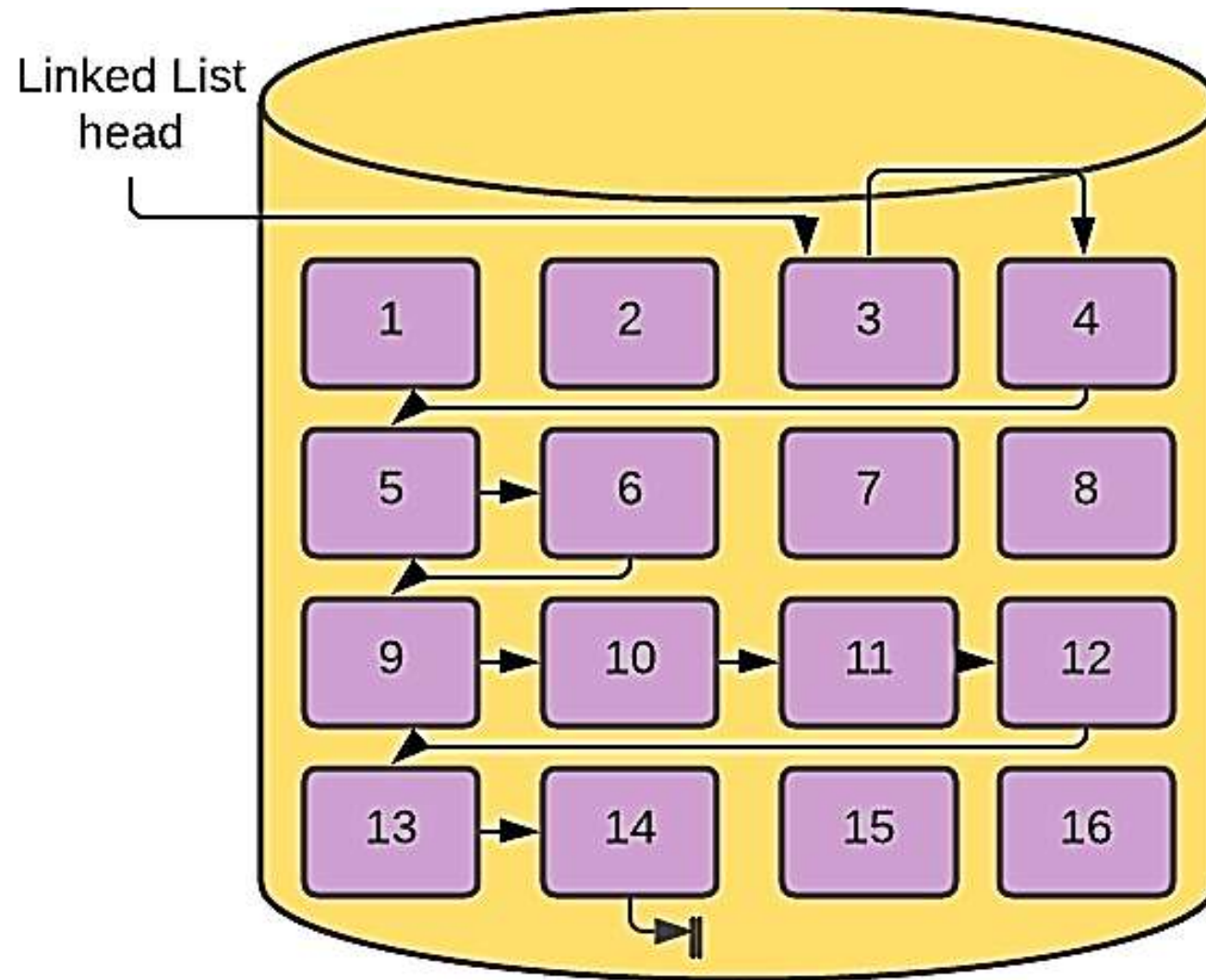


Fig: Linked List Free Space Management Technique

Linked List :

Advantage of the Linked list :

- In this method, available space is used efficiently.
- As there is no size limit on a linked list, a new free space can be added easily.

Disadvantages :

- In this method, the overhead of maintaining the pointer appears.
- The Linked list is not efficient when we need to reach every block of memory.

Virtual Memory

- Virtual memory is the separation of user logical memory from physical memory. This separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available.
- It makes the task of programming much easier, because the programmer no longer needs to worry about the amount of physical memory available, or about what code can be placed in overlays, but can concentrate instead on the problem to be programmed.
- It is the process of increasing the apparent size of a computer's RAM by using a section of the hard disk storage as an extension of RAM.

Definition of Virtual Memory

- Virtual memory is a memory management capability of an operating system (OS) that uses hardware and software to allow a computer to compensate for physical memory shortages by temporarily transferring data from random access memory (RAM) to disk storage.

Definition of Paging

(2M)

- Paging refers to the transfer of memory pages from physical memory to disk and vice versa.
- Virtual memory uses a technique called demand paging for its implementation.
- Logical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available.

Paging

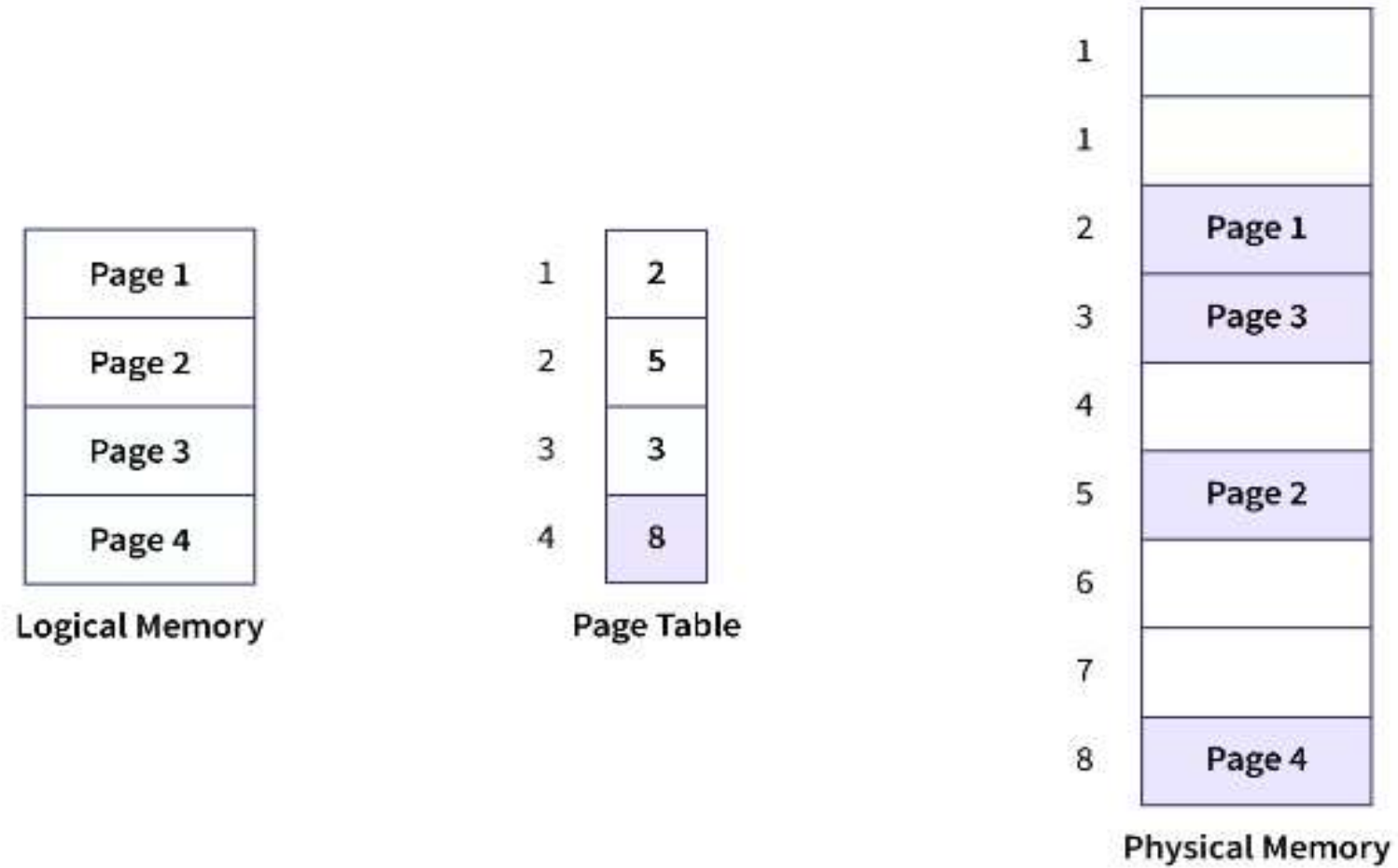
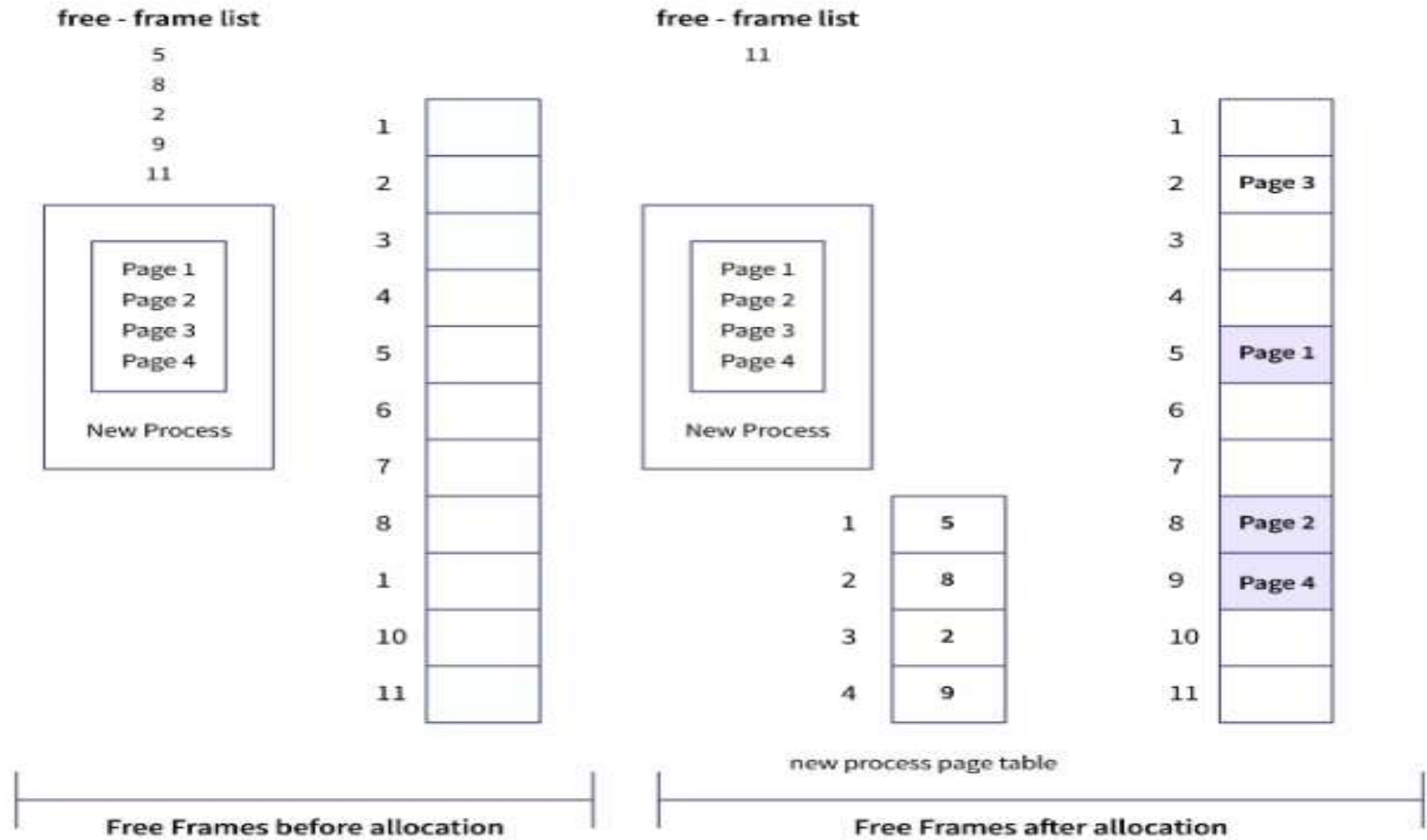


Fig: Paging Model of Logical and Physical Memory

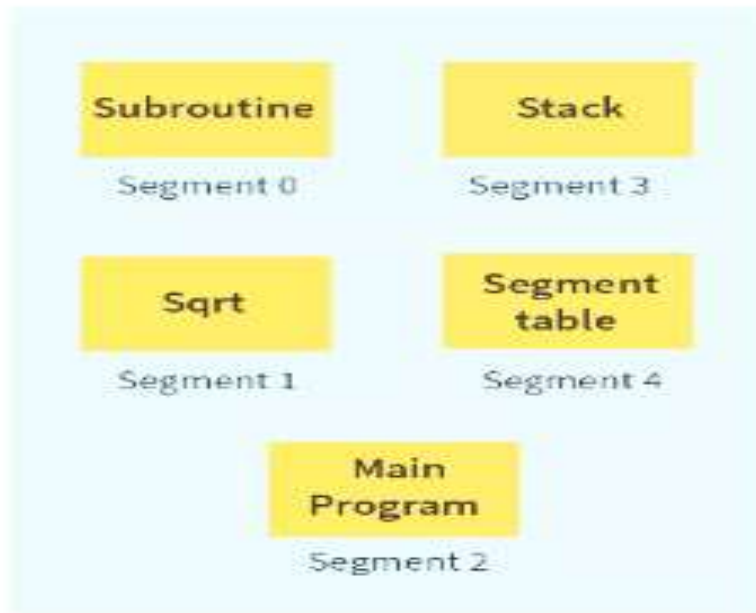


Segmentation

- Segmentation divides the user program and the secondary memory into uneven-sized blocks known as **Segments**. Segmentation can be divided into two types namely - Virtual Memory Segmentation and Simple Segmentation.
- Segmentation divides processes into smaller subparts known as modules. The divided segments need not be placed in contiguous manner. Since there is no contiguous memory allocation, internal fragmentation does not take place. Segment size depends on the purpose of the segment in the program.

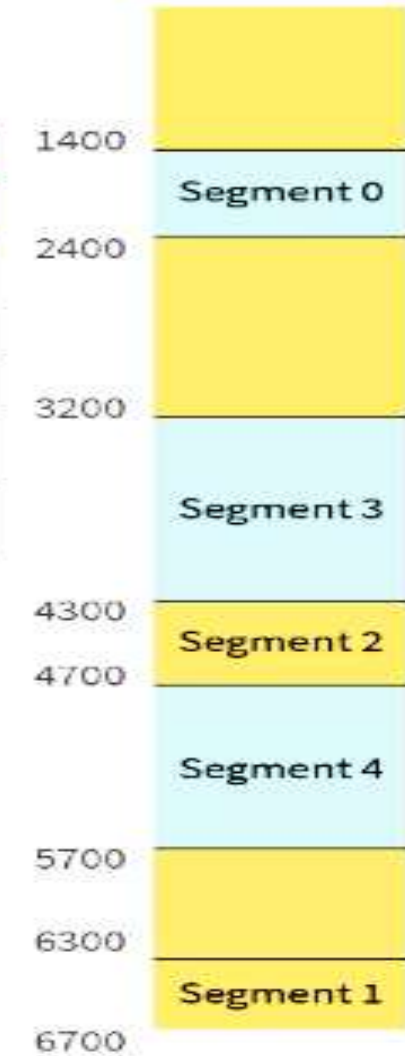
Segment Table and its Uses

- Segment Table is used to store the information of all segments of the process. As we know, the CPU generates a logical address and for its conversion to the physical address, a segment table is used.
- The mapping of a two-dimensional Logical address to the one-dimensional Physical address is done using the segment table.
- The Segment Table has two components:
 - 1.Segment Base:** The segment base is also known as the base address of a segment. The segment base contains the starting physical address of the segments residing in the memory.
 - 2.Segment Limit:** The segment limit is also known as segment offset. The segment contains the specific length of the segment.



Logical Address Space

Segment Table		
	Limit	Base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700



Physical Memory

Fig: Segmentation with it's segment table

Segmentation: Advantages

- No internal fragmentation is there in segmentation.
- Segment Table is used to store the records of the segments. The segment table itself *consumes small memory* as compared to a page table in paging.
- Segmentation provides *better CPU utilization* as an entire module is loaded at once.
- The Segment size is specified by the user but in Paging, the hardware decides the page size.
- Segmentation can be used to separate the security procedures and data.
- Segmentation is near to the user's view of physical memory. Segmentation allows users to partition the user programs into modules. These modules are nothing but the independent codes of the current process.

Segmentation: Disadvantages

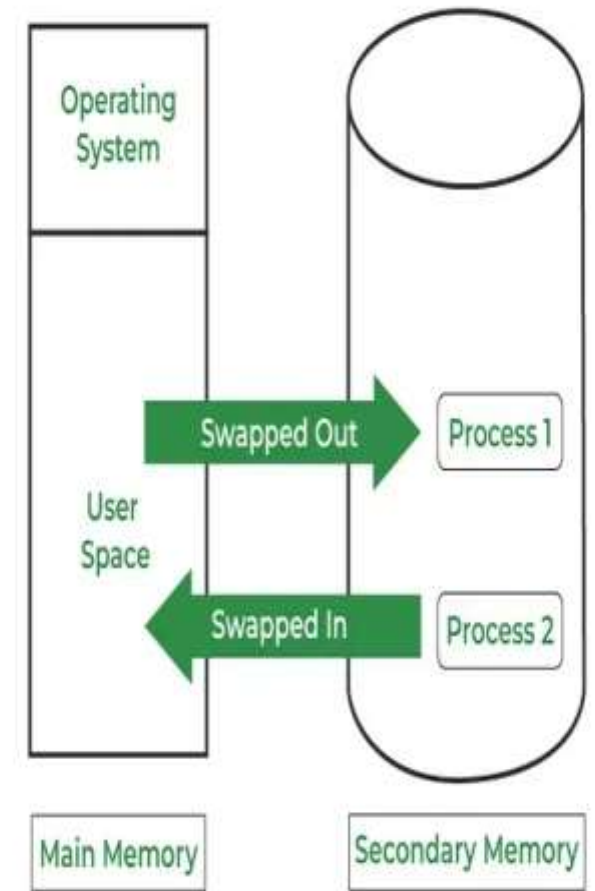
- During the swapping of processes the free memory space is broken into small pieces, which is a major problem in the segmentation technique.
- Time is required to fetch instructions or segments.
- The swapping of segments of unequal sizes is not easy.
- There is an ***overhead of maintaining a segment table*** for each process as well.
- When a process is completed, it is removed from the main memory. After the execution of the current process, the unevenly sized segments of the process are removed from the main memory. Since the segments are of uneven length it creates unevenly sized holes in the main memory. These holes in the main memory may remain unused due to their very small size.

Difference between Segmentation and Paging (Any 6 points)

Paging	Segmentation
It divides the physical memory into frames and program's address space into same size pages.	It divides the Computer's physical memory and program's address space into segments.
Page is always of fixed block size.	Segment is of variable size.
The size of the page is specified by the hardware.	The size of the segment is specified by the user.
It may lead to internal fragmentation as the page is of fixed block size.	It may lead to External fragmentation as the memory is filled with the variable sized blocks.
Page table is used to map pages with frames from memory.	Segment table is used to map segments with physical memory.
Page table contains page number and frame number.	Segment table contains segment number, length of segment and base address of segment from memory.
Invisible to Programmer	Visible to programmer
Paging consist of Static linking & dynamic loading	Segment consist of Dynamic Linking & Dynamic Loading
A page is of physical unit	A segment is of logical unit

Swapping

- Swapping in OS is one of those technique which fulfill the goal of maximum utilization of CPU and memory management by **swapping in** and **swapping out** processes from the main memory. **Swap in** removes the process from hard drive(secondary memory) and **swap out** removes the process from RAM(main memory).
- Swapping is a memory management technique in which a process is temporarily moved from main memory (RAM) to secondary storage (disk) and vice versa.
- This allows the operating system to manage limited RAM effectively and run multiple processes concurrently in a multiprogramming environment.
- Reduces memory fragmentation by moving processes in and out of RAM.
- Allows large processes to run even if they exceed available RAM.



Swapping

- Frees RAM temporarily by suspending inactive or waiting processes.
- Supports virtual memory to give processes more memory than physically available.

Example: RAM has 4 GB with Process A and Process B running, each using 2 GB. When Process C, which also requires 2 GB, needs to run, there is no free memory. The operating system swaps Process A out to the disk and loads Process C into RAM. After Process C finishes, Process A is brought back into memory to continue execution.

There are two important concepts in the process of swapping which are as follows:

1. Swap In
2. Swap Out

1. Swap In: The method of removing a process from secondary memory (**Hard Drive**) and restoring it to the main memory (**RAM**) for execution is known as the Swap In method.

2. Swap Out: It is a method of bringing out a process from the main memory(**RAM**) and sending it to the secondary memory(**hard drive**) so that the processes with higher priority or more memory consumption will be executed known as the Swap Out method.

Advantages of Swapping

- Swapping in **OS** helps in achieving the goal of **Maximum CPU Utilization**.
- Swapping ensures proper memory availability for every process that needs to be executed.
- Swapping helps avoid the problem of process starvation means a process should not take much time for execution so that the next process should be executed.
- **CPU** can perform various tasks simultaneously with the help of swapping so that processes do not have to wait much longer before execution.
- Swapping ensures proper **RAM**(main memory) utilization.
- Swapping in **OS** is an economical process.
- Swapping method can be applied on **priority-based** process scheduling where a **high priority** process is swapped in and a **low priority** process is swapped out which improves the performance.

Disadvantages of Swapping

- If the system deals with **power-cut** during bulky swapping activity then the user may lose all the information which is related to the program.
- If the swapping method uses an algorithm that is not up to the mark then the number of page faults can be increased and therefore this decreases the complete performance.
- There may be inefficiency in a case when there is some common resource used by the processes that are participating in the swapping process.

Fragmentation

Fragmentation refers to a process of information storage where the memory space of the system is used inadequately, thus reducing the overall efficiency or ability or both (sometimes).

➤ **Types of Fragmentation**

- External Fragmentation
- Internal Fragmentation

Internal Fragmentation

- Whenever a memory block gets allocated with a process, and in case the process happens to be smaller than the total amount of requested memory, a free space is ultimately created in this memory block.
- And due to this, the memory block's free space is unused.
- This is what causes internal fragmentation.



Fig: Internal Fragmentation

External Fragmentation

- External fragmentation occurs whenever a method of dynamic memory allocation happens to allocate some memory and leave a small amount of unusable memory.
- The total quantity of the memory available is reduced substantially in case there's too much external fragmentation.
- So, there's enough memory space in order to complete a request, and it is not contiguous.
- Thus, it is known as external fragmentation.

Compaction

- Compaction is a technique to collect all the free memory present in form of fragments into one large chunk of free memory, which can be used to run other processes.
- It does that by moving all the processes towards one end of the memory and all the available free space towards the other end of the memory so that it becomes contiguous.
- It is not always easy to do compaction.

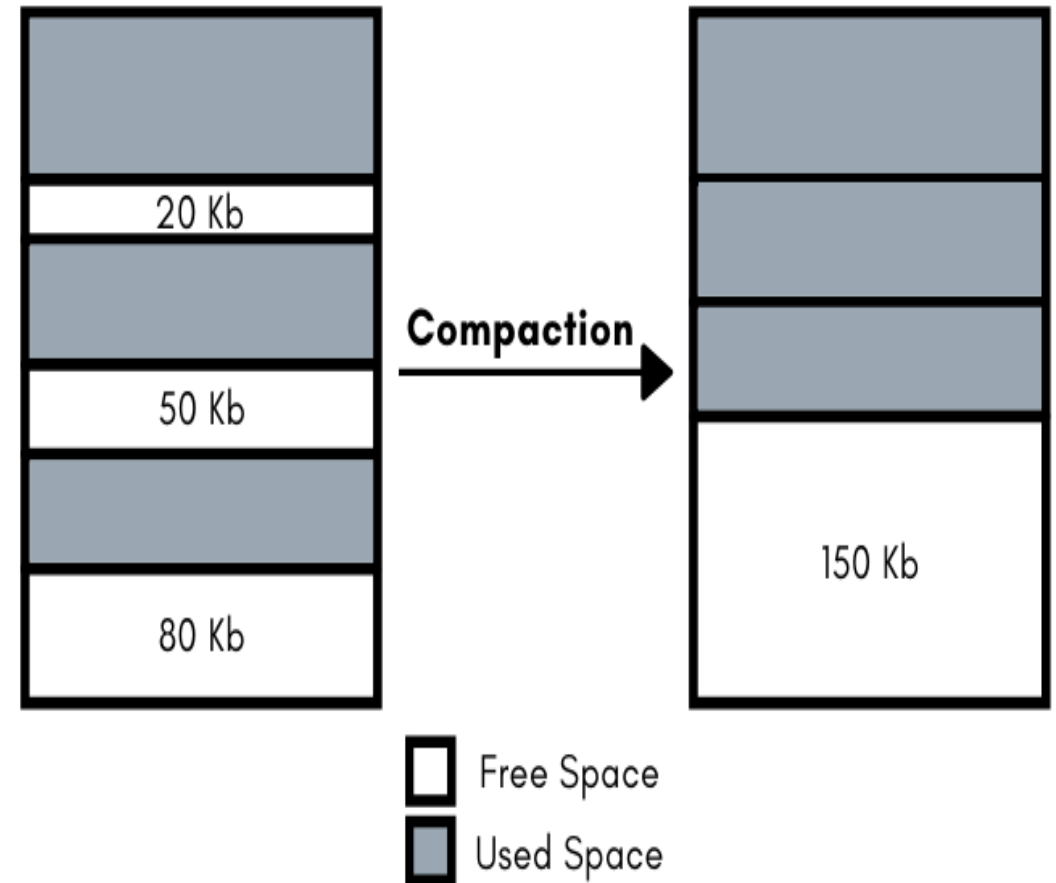


Fig: Compaction

Compaction

Advantages of Compaction

- Reduces external fragmentation.
- Make memory usage efficient.
- Memory becomes contiguous.
- Since memory becomes contiguous more processes can be loaded to memory.
- Improves memory utilization.

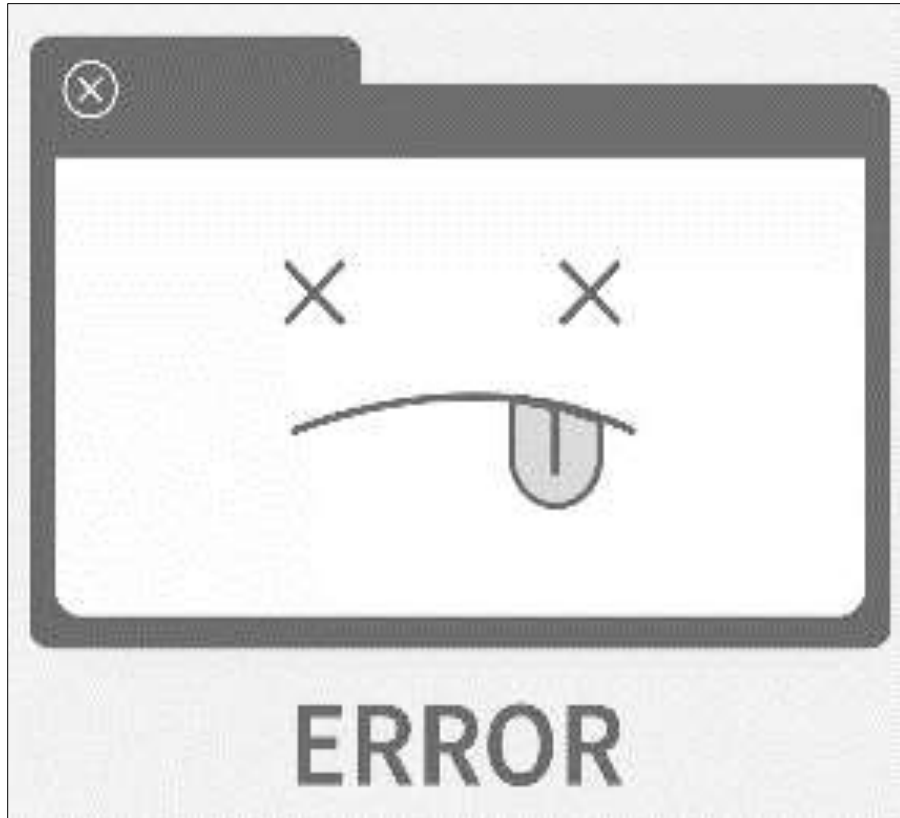
Disadvantages of Compaction

- A huge amount of time is wasted in performing compaction.
- CPU sits idle for a long time.
- Not always easy to perform compaction.
- It may cause deadlocks since it disturbs the memory allocation process.

Difference between Internal and External fragmentation

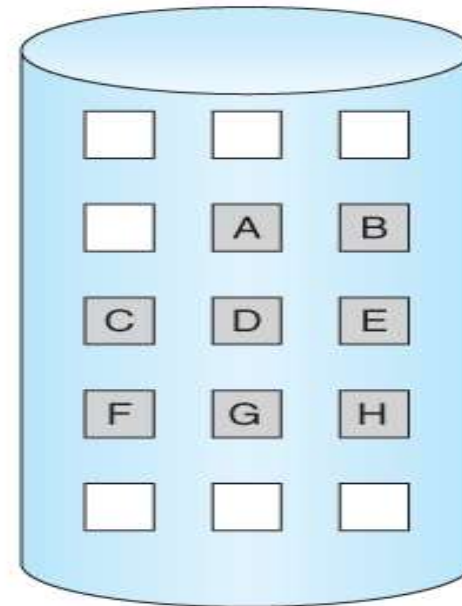
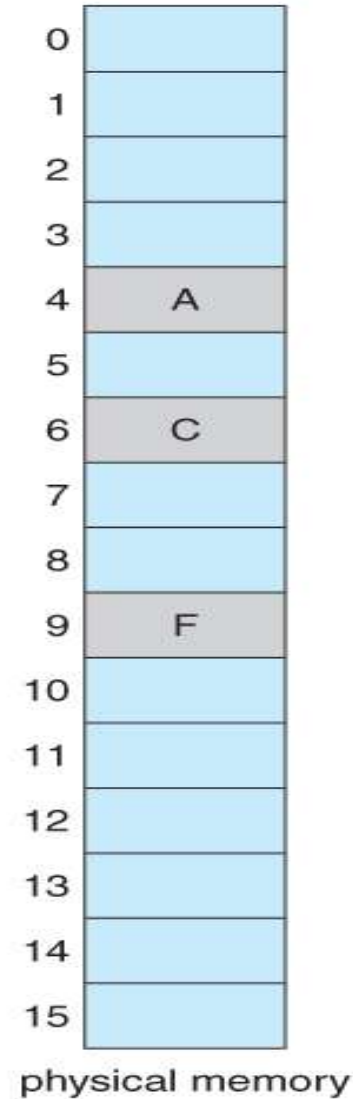
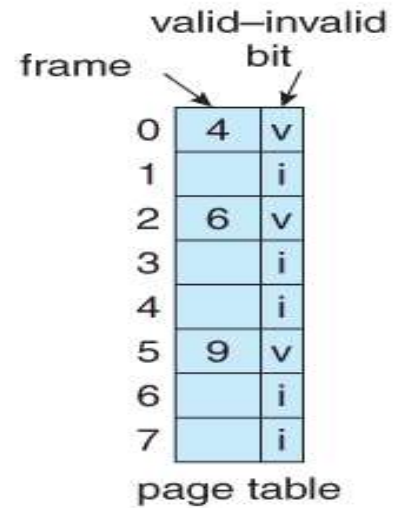
Internal Fragmentation	External Fragmentation
Unused memory blocks are allocated.	Available memory blocks are non-contiguous.
Memory is divided into fixed-size blocks.	Memory is divided into varying-size blocks.
Occurs when allocated space is bigger than needed space.	Occurs at the removal of a process from memory.
Best fit block search is the solution.	Compaction is the solution.

Page Fault



- A **page fault** is trap that occurs when the requested page is not loaded into the memory.
- In other words, when a **program tries** to access a chunk of memory that does not exist in physical memory (main memory) causes a page fault.
- To distinguish between the pages that are on the disk, a valid and invalid bit is used.

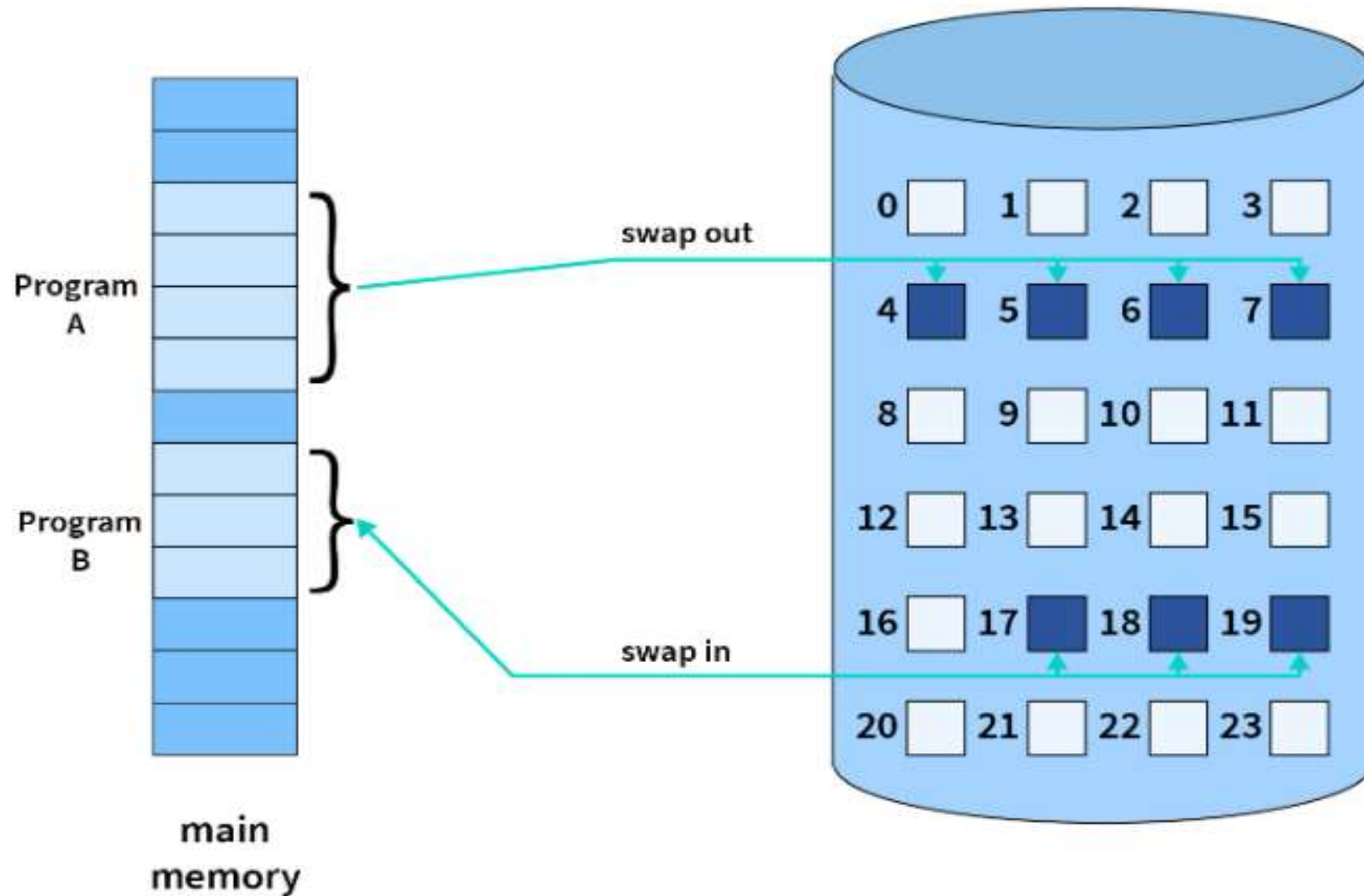
Page Table When some Pages are found in Main Memory



What is Demand Paging?

- Imagine that you are scrolling through an e-commerce platform and like some clothes.
- However, since you don't have space in your wardrobe, you save it in your Wishlist and decide to buy them only when needed.
- As compared to this process, demand paging is a process that keeps pages of a process that are infrequently used in secondary memory, and pulls them only when required to satisfy the demand.
- As a result, when a context switch happens, the OS begins executing the new program after loading the first page and only retrieves the application's referenced pages.
- If the software addresses a page that is not available in the main memory because it was swapped, the processor deems it as an invalid memory reference.

What is Demand Paging?



Demand Paging: Advantages & Disadvantages

➤ Advantages:

- Memory can be put to better use.
- If we use demand paging, then we can have a large virtual memory.
- No limit on degree of multiprogramming.

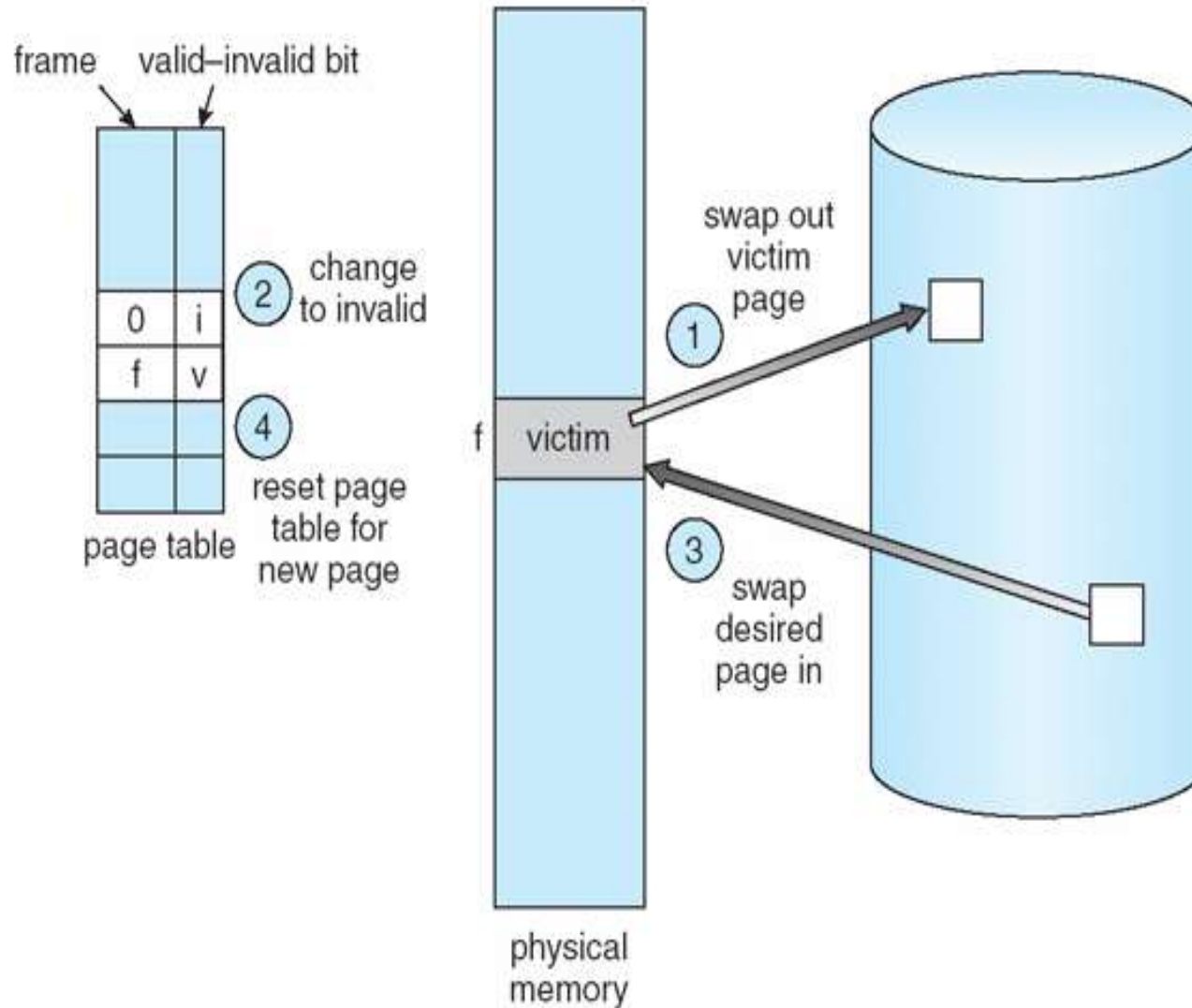
➤ Disadvantages:

- Internal fragmentation is a possibility with demand paging.
- It takes longer to access memory.

Page Replacement Algorithms

- Page replacement is needed in the operating systems that use virtual memory using Demand Paging. As we know that in Demand paging, only a set of pages of a process is loaded into the memory. This is done so that we can have more processes in the memory at the same time.
- When a page that is residing in virtual memory is requested by a process for its execution, the Operating System needs to decide which page will be replaced by this requested page.
- This process is known as page replacement and is a vital component in virtual memory management.

Steps for Page Replacement



1. Find the location of the desired page on the disk.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to the disk; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Restart the user process.

Page Replacement Algorithms in Operating Systems

1. First In First Out (FIFO)
2. Optimal Page
3. Least Recently Used (LRU)

First In First Out (FIFO)

- FIFO algorithm is the simplest of all the page replacement algorithms.
- In this, we maintain a queue of all the pages that are in the memory currently.
- The oldest page in the memory is at the front-end of the queue and the most recent page is at the back or rear-end of the queue.
- Whenever a page fault occurs, the operating system looks at the front-end of the queue to know the page to be replaced by the newly requested page.
- It also adds this newly requested page at the rear-end and removes the oldest page from the front-end of the queue.

First In First Out (FIFO)

- **Example:** Consider the page reference string as 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Let's try to find the number of page faults:
- Initially, all of the slots are empty so page faults occur at 3,1,2.

Page faults = 3

- When page 1 comes, it is in the memory so no page fault occurs.

Page faults = 3

- When page 6 comes, it is not present and a page fault occurs. Since there are no empty slots, we **remove the front of the queue, i.e 3.**

Page faults = 4

- When page 5 comes, it is also not present and hence a page fault occurs. The front of the queue i.e **1 is removed.**

Page faults = 5

- When page 1 comes, it is not found in memory and again a page fault occurs. The front of the queue i.e **2 is removed.**

Page faults = 6

- When page 3 comes, it is again not found in memory, a page fault occurs, and **page 6 is removed** being on top of the queue

Total page faults = 7

First In First Out (FIFO)

➤ Advantages

- Simple to understand and implement
- Does not cause more overhead

➤ Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.

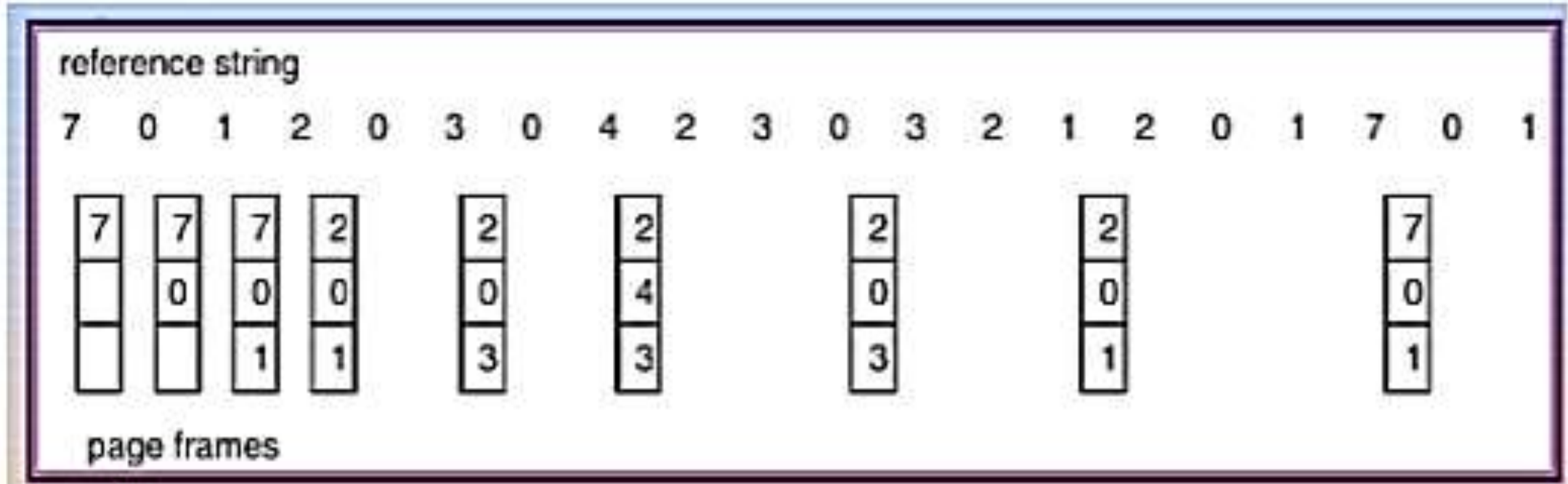
Optimal Page Replacement

- Optimal page replacement is the best page replacement algorithm as this algorithm results in the least number of page faults.
- In this algorithm, the pages are replaced with the ones that will not be used for the longest duration of time in the future.
- In simple terms, the pages that will be referred farthest in the future are replaced in this algorithm.

Example:-

Reference string:- 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

Number of frames: 3



Number of page fault:-09

Number of page hit:11

Optimal Page Replacement

➤ Advantages

- Excellent efficiency
- Less complexity
- Easy to use and understand
- Simple data structures can be used to implement
- Used as the benchmark for other algorithms

➤ Disadvantages

- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Least Recently Used (LRU) Page Replacement

- The Least Recently Used (LRU) page replacement policy replaces the page that has not been used for the longest period of time.
- LRU replacement associates with each page the time of that page's last use.
- When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.
- The LRU policy is often used as a page-replacement algorithm and is considered to be good.
- An LRU page-replacement algorithm may require substantial hardware assistance

Least Recently Used (LRU) Page Replacement

➤ Counters:

- In the simplest case, we associate with each page-table entry a time-of-use field and add to the CPU a logical clock or counter.
- The clock is incremented for every memory reference.
- Whenever a reference to a page is made, the contents of the clock register are copied to the time-of-use field in the page-table entry for that page.
- In this way, we always have the "time" of the last reference to each page. We replace the page with the smallest time value.

Least Recently Used (LRU) Page Replacement

➤ Stack:

- Another approach to implementing LRU replacement is to keep a stack of page numbers.
- Whenever a page is referenced, it is removed from the stack and put on the top.
- In this way, the most recently used page is always at the top of the stack and the least recently used page is always at the bottom

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
 (Frame size have not mentioned in question so assume frame size as 3 or 4)

LRU: Assume frame size=3

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		4	4	4	0			1		1	*	1		*
	0	0	0	*	0	*	0	0	3	3	*		3		0		0	*	
		1	1		3		3	2	2	2		*	2	*	2		7		

Page Fault=12

Assume frame size=4

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	7		3		3		*		*		3				7		
	0	0	0	*	0	*	0			*			0		*		0	*	
		1	1		1		4						1			*	1		*
			2		2		2	*				*	2	*			2		

Page fault=08

Least Recently Used (LRU) Page Replacement

➤ Advantages

- It is open for full analysis
- Often more efficient than other algorithms

➤ Disadvantages

- It requires additional data structures to be implemented
- More complex
- High hardware assistance is required

Thank You!